

Why Sliding Window is Used

• Page 2 / 10 •



- ① Brute Force approach recalculates every subarray / substring from scratch. For each starting index, it scans again up to k elements. This leads to many repeated additions and usually takes $O(n^2)$.
- ② Sliding Window reuses previous work by removing the element that goes out and adding the new element that comes in. Thus, each step does $O(1)$ work and the whole process becomes $O(n)$.

Brute Force vs Sliding Window

Brute Force

- Recalculate every window sum from scratch.
- Time Complexity: $O(n*k)$ or $O(n^2)$

Sliding Window

- Reuse previous window sum.
- Time Complexity: $O(n)$



Optimize & Save Time!

Example

Array: [2, 1, 5, 1, 3, 2] $k = 3$

- ① Brute Force (recalculate every time)

[2, 1, 5, 1, 3, 2] → $2+1+5 = 8$

[2, 1, 5, 1, 3, 2] → $1+5+1 = 7$

[2, 1, 5, 1, 3, 2] → $5+1+3 = 9$

[2, 1, 5, 1, 3, 2] → $1+3+2 = 6$

Repeated recalculation of sums.

- ② Sliding Window (reuse previous work)

Window 1: [2, 1, 5] sum = $2+1+5 = 8$

Window 2: [1, 5, 1] $8 - 2 + 1 = 7$

Window 3: [5, 1, 3] $7 - 1 + 3 = 9$

Window 4: [1, 3, 2] $9 - 5 + 2 = 6$

Key Formula

$$\text{new_window_sum} = \text{previous_window_sum} - \text{outgoing} + \text{incoming}$$

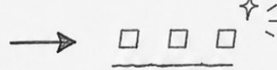
Time Complexity

Brute Force : $O(n*k)$ or $O(n^2)$

Sliding Window : $O(n)$



Many calculations



Smart optimization!

≡ Page 2 ≡

When to Use Sliding Window

• Page 3 / 10 •



Use it when...

- We are dealing with a contiguous subarray.
- We are dealing with a contiguous substring.
- There is a fixed size k .
- We need the longest substring / subarray.
- We need the shortest subarray.
- There is an "at most k " (or similar) condition.
- We need count / frequency inside a window.



Common Keywords

subarray	→	sliding window
substring	→	sliding window
size k	→	fixed window
longest	→	variable window
smallest / minimum	→	variable window
no repeating characters	→	sliding window + set / map



How to Identify

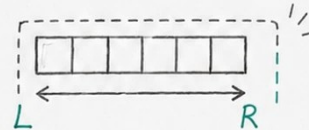
- ① Is the problem on arrays / strings?
- ② Is the range continuous?
- ③ Do we need maximum / minimum / length / count?
- ④ Can we expand and shrink a range instead of checking every range?



Mini Example

Find longest substring without repeating characters.

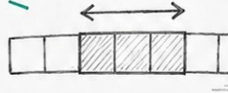
→ This is a variable-size sliding window problem.



≡ Page 3 ≡

Question 1: Maximum Sum Subarray of Size K

• Page 7 / 10 •



★ Problem:

Given an array of integers `arr` and an integer `k`, find the maximum sum of any contiguous subarray of size `k`.

★ Example:

`arr = [2, 1, 5, 1, 3, 2]`, `k = 3`

Output = 9

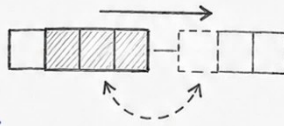


Windows of size `k=3`:

2	1	5	1	3	2	→ [2, 1, 5] = 8
2	1	5	1	3	2	→ [1, 5, 1] = 7
2	1	5	1	3	2	→ [5, 1, 3] = 9 ★ Max!
2	1	1	1	3	2	→ [1, 3, 2] = 6

Approach

- ① Compute the sum of the first window of size `k`.
- ② Slide the window one step at a time.
- ③ Subtract the outgoing element (leftmost) from the sum.
- ④ Add the incoming element (rightmost) to the sum and update the maximum.



Why sliding window?

Brute force checks every window from scratch in $O(n \cdot k)$ time.
Sliding window reuses the previous sum and updates in $O(1)$ time.

Update Formula

$$\text{new_sum} = \text{old_sum} - \text{outgoing} + \text{incoming}$$

↑
element
leaving

↑
element
entering



Interview Tip ★

This is the easiest and most basic sliding-window problem.
Make sure you are comfortable with it before moving to harder ones!

≡ Page 7 ≡

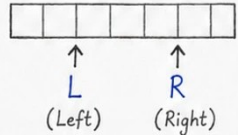


Variable-Size Sliding Window Pattern

• Page 5 / 10 •



- ① Definition : In variable-size sliding window, the size of the window changes according to a condition.
- ② Explanation : We use two pointers — left and right.
 - Right pointer expands the window.
 - Left pointer shrinks the window when the condition is satisfied or violated.



General Idea ..

- ① Expand right pointer.
- ② Add current element to window.
- ③ While condition is satisfied or broken, shrink from left.
- ④ Update answer.



Example : Smallest subarray with sum $\geq$ target

target = 7
arr = [2, 3, 1, 2, 4, 3]

Dry Run :

- ① Expand R to get sum $\geq$ target.
 sum = 8 (≥ 7)
 window = [2, 3, 1, 2]
 min_len = 4
- ② Shrink from left while sum $\geq$ target.
 sum = 6 (< 7)
 window = [3, 1, 2]
- ③ Continue expanding R.
 sum = 7 (≥ 7)
 window = [4, 3]
 min_len = 2

Answer: 2

Python Template

```
def min_subarray_len(target, arr):
    left = 0
    window_sum = 0
    min_len = float('inf')
    for right in range(len(arr)):
        window_sum += arr[right]
        while window_sum >= target:
            min_len = min(min_len, right-left+1)
            window_sum -= arr[left]
            left += 1
    return 0 if min_len == float('inf') else min_len
```

Key Note ★

Used for longest/shortest substring or subarray problems.



≡ Page 5 ≡

Fixed-Size Sliding Window Pattern

• Page 4 / 10 •

Definition: Fixed-size window is used when k is given.
We always consider subarrays of size exactly k .

Steps ..

- Build the first window of size k .
- Store answer.
- Slide by 1 position.
- Add incoming element.
- Remove outgoing element.
- Update answer.

Formula

$window_sum += arr[i]$
 $window_sum -= arr[i-k]$

Dry Run: $arr = [2, 1, 5, 1, 3, 2]$, $k = 3$

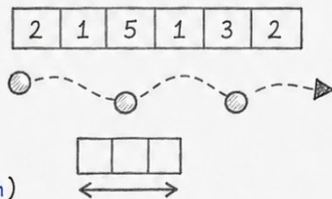
	2	1	5	1	3	2
Window 1:	2	1	5			
Window 2:	2	1	5	1		
Window 3:	2	1	5	1	3	
Window 4:	2	1	1	3	2	
	Sum = 8	Sum = 7	Sum = 9	Sum = 6		

Note

Best for
maximum/
average/sum/
count of
subarray of
size k .

Template ..

```
def max_sum_subarray(arr, k):
    window_sum = sum(arr[:k])
    max_sum = window_sum
    for i in range(k, len(arr)):
        window_sum += arr[i]
        window_sum -= arr[i-k]
        max_sum = max(max_sum, window_sum)
    return max_sum
```



≡ Page 4 ≡

Question 2: Longest Substring Without Repeating Characters

• Page 9 / 10 •

Problem:

Given a string s , find the length of the longest substring without repeating characters.

Example: $s = "abcabcb"$, output = 3 because "abc" is the longest.

Approach:

Use a variable-size sliding window with a set.

- Expand the window by moving right.
- If a duplicate appears, shrink the window from left until it becomes valid again.

Python Code:

```
def length_of_longest_substring(s):
    left = 0
    seen = set()
    max_len = 0
    for right in range(len(s)):
        while s[right] in seen:
            seen.remove(s[left])
            left += 1
        seen.add(s[right])
        max_len = max(max_len, right-left+1)
    return max_len
```

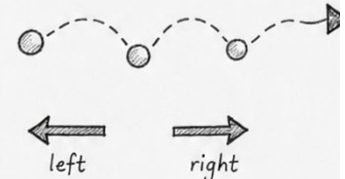
`print(length_of_longest_substring("abcabcb"))`

Dry Run on $s = "abcabcb"$:

Index :	0	1	2	3	4	5	6	7
Char :	a	b	c	a	b	c	b	b
Window:	a	ab	abc	bca	cab	abc	bc	cb
left :	0	0	0	1	2	3	4	5
right :	0	1	2	3	4	5	6	7

Output: 3

Time $O(n)$, Space $O(n)$



≡ Page 9 ≡

Full Code: Maximum Sum Subarray of Size K

• Page 8 / 10 •



Python Solution

```
def max_sum_subarray(arr, k):
    if len(arr) < k:
        return -1
```

```
    window_sum = sum(arr[:k])
```

```
    max_sum = window_sum
```

```
    for i in range(k, len(arr)):
```

```
        window_sum += arr[i]
```

```
        window_sum -= arr[i-k]
```

```
        max_sum = max(max_sum, window_sum)
```

```
    return max_sum
```

```
arr = [2, 1, 5, 1, 3, 2]
```

```
k = 3
```

```
print(max_sum_subarray(arr, k))
```



≡ Page 8 ≡

Output: 9

Code Explanation

- First, compute the sum of the first window of size k.
- Slide the window one element at a time by adding the new element and removing the old one.
- Update the maximum sum if the current window sum is greater.

Complexity

- Time : $O(n)$
- Space : $O(1)$

Question 3: Minimum Size Subarray Sum

• Page 10 / 10 •



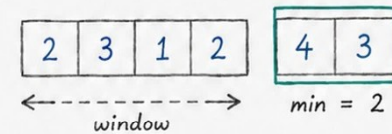
Problem Statement ..

Given an array of positive integers and a target, return the minimum length of a contiguous subarray whose sum is greater than or equal to target.

Example ..

target = 7, arr = [2, 3, 1, 2, 4, 3]

Output = 2 because [4, 3].



Approach ..

Use variable-size sliding window. Keep expanding the right pointer to increase the sum. Once sum $\geq$ target, update the minimum length and shrink the window from the left to see if we can get a smaller window.



Python Code ..

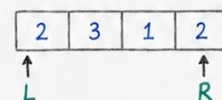
```
def min_subarray_len(target, arr):
    left = 0
    window_sum = 0
    min_len = float('inf')
    for right in range(len(arr)):
        window_sum += arr[right]
        while window_sum >= target:
            min_len = min(min_len, right-left+1)
            window_sum -= arr[left]
            left += 1
    return 0 if min_len == float('inf') else min_len

print(min_subarray_len(7, [2, 3, 1, 2, 4, 3]))
```

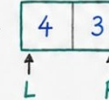
Output: 2

Time $O(n)$,
Space $O(1)$

Dry Run (target = 7) ..



sum = 8 $\geq$ 7 $\rightarrow$ update min = 4
shrink from left...



sum = 7
min = 2



≡ Page 10 ≡

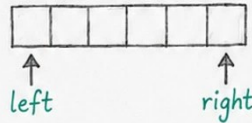
Common Mistakes and Dry Run Method

• Page 6 / 10 •



① Common Mistakes

- forgetting to remove outgoing element
- wrong window size check
- updating answer at wrong time
- not shrinking window in while loop
- confusing fixed and variable window
- not handling duplicates with set/map properly
- off-by-one errors in $\text{length} = \text{right} - \text{left} + 1$



② How to Dry Run

- ① Write array/string
- ② Mark left and right
- ③ Track sum/frequency/set
- ④ Write current answer
- ⑤ Move pointers carefully

Golden Formulae ☆

$\text{length} = \text{right} - \text{left} + 1$

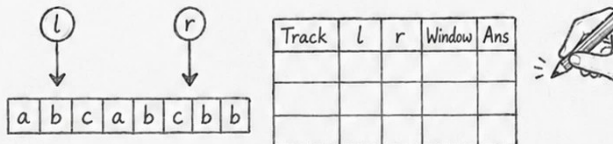
$\text{incoming} = \text{arr}[\text{right}]$

$\text{outgoing} = \text{arr}[\text{left}]$ ☆

Example : Longest Substring Without Repeating Characters
String = "abcabcbb"

Step	r	Char	Action	Window	l	Ans
0	0	a	add	a	0	1
1	1	b	add	ab	0	2
2	2	c	add	abc	0	3
3	3	a	dup → shrink l	bca	1	3
4	4	b	dup → shrink l	cab	2	3
5	5	c	dup → shrink l	abc	3	3
6	6	b	dup → shrink l	cb	5	3
7	7	b	dup → shrink l	b	7	3

Final Answer = 3



≡ Page 6 ≡

Sliding Window

• Page 1 / 10 •



- ① Sliding Window is an optimization technique used for arrays and strings to solve problems involving subarrays or substrings.
- ② Instead of recalculating every subarray again and again, we maintain a **window** and slide it forward.
- ③ It is very useful for **contiguous** elements.
- ④ It often reduces time complexity from $O(n^2)$ to $O(n)$.

Basic Idea ..

Example array :

2	1	5	1	3	2
---	---	---	---	---	---

 $k = 3$

- ①

2	1	5	1	3	2
---	---	---	---	---	---

 → $[2, 1, 5] = 8$
- ②

2	1	5	1	3	2
---	---	---	---	---	---

 → $[1, 5, 1] = 7$
- ③

2	1	5	1	3	2
---	---	---	---	---	---

 → $[5, 1, 3] = 9$
- ④

2	1	5	1	3	2
---	---	---	---	---	---

 → $[1, 3, 2] = 6$

Maximum sum = 9



≡ Page 1 ≡

Key Note ☆

Reuse the previous window result instead of starting from zero ☆